ECE 2011
Introduction to Computer Engineering
Spring 2026

Lecture 17
03-31-2026

Linghao Song
Yale ECE

Assignment 4

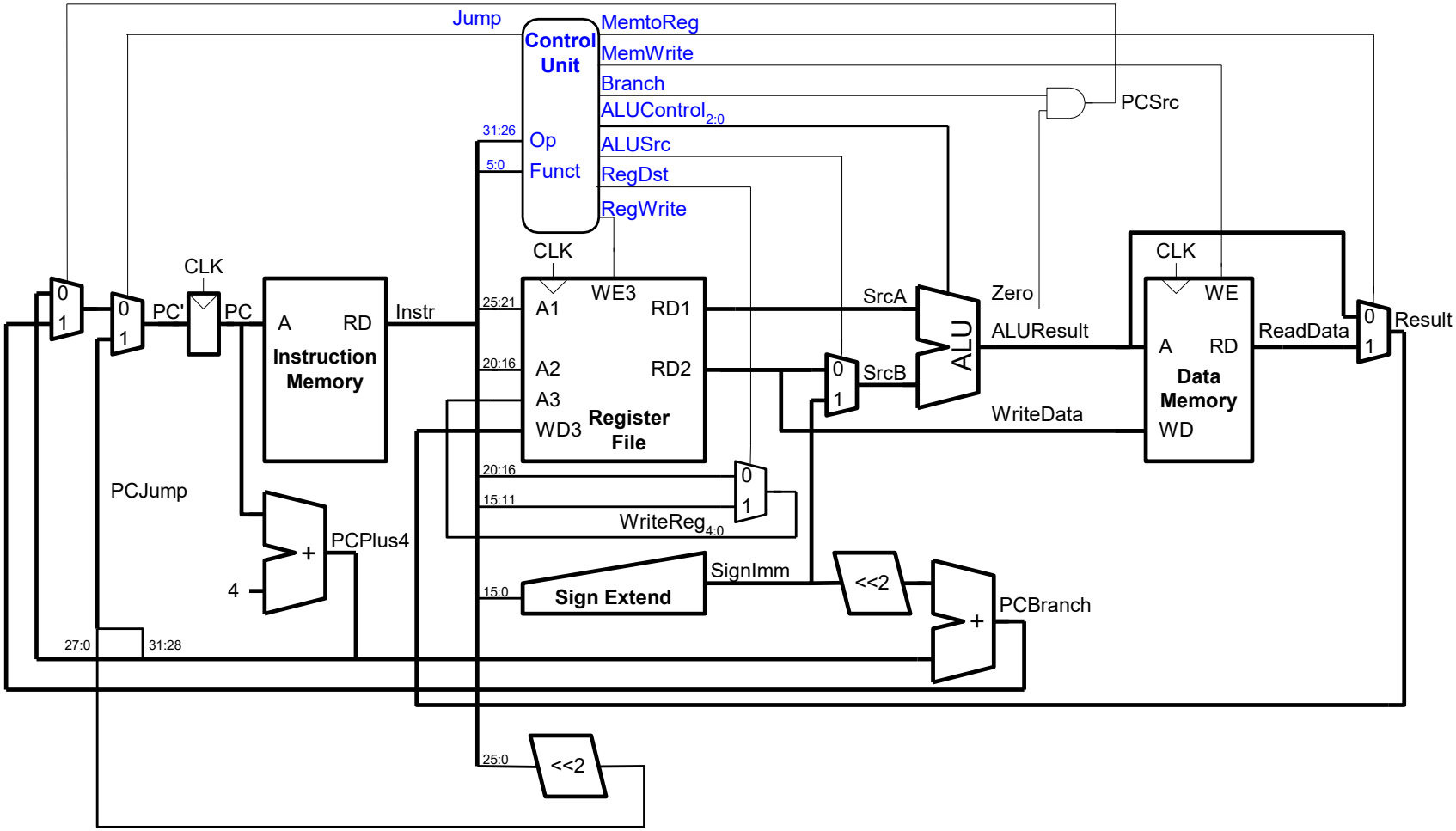
- Sign up
- https://docs.google.com/spreadsheets/d/1ezydqTY5wrIBKDSLpwee_e9xpLkA4WnUzkj0U1XD2fU/edit?gid=0#gid=0

Assignment 4 Sign-Up Sheet: Paper Presentation Groups.

Topic	Paper	Member 1	Member 2	Member 3	Member 4	Member 5	(use c
Topic 1: Evolution of Google TPU Architectures	TPU (ISCA 2017)						
Topic 1: Evolution of Google TPU Architectures	TPU v4 (ISCA 2023)						
Topic 2: Efficient LLM Inference on Specialized Hardwar	Splitwise (ISCA 2024)						
Topic 2: Efficient LLM Inference on Specialized Hardwar	FlightLLM (FPGA 2024)						
Topic 3: Vision Transformer Acceleration	HeatViT (HPCA 2023)						
Topic 3: Vision Transformer Acceleration	ViTCoD (HPCA 2023)						
Topic 4: FPGA CAD and Compilation	AutoBridge (FPGA 2021)						
Topic 4: FPGA CAD and Compilation	ARIES (FPGA 2025)						
Topic 5: FPGA Architectures for Scientific Compute	Callipepla (FPGA 2023)						
Topic 5: FPGA Architectures for Scientific Compute	Du et al (FPGA 2022)						

Short name	Full title	Conference	Year	DOI / URL
TPU (ISCA 2017)	In-Datacenter Performance Analysis of a Tensor Processing Unit	ISCA	2017	https://arxiv.org/abs/1704.04760
TPU v4 (ISCA 2023)	TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddin	ISCA	2023	https://arxiv.org/abs/2304.01433
Splitwise (ISCA 2024)	Splitwise: Efficient Generative LLM Inference Using Phase Splitting	ISCA	2024	https://doi.org/10.1109/ISCA59077.2024.00019
FlightLLM (FPGA 2024)	FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs	FPGA	2024	https://arxiv.org/abs/2401.03868
HeatViT (HPCA 2023)	HeatViT: Hardware-Efficient Adaptive Token Pruning for Vision Transformers	HPCA	2023	https://doi.org/10.1109/HPCA56546.2023.10071047
ViTCoD (HPCA 2023)	ViTCoD: Vision Transformer Acceleration via Dedicated Algorithm and Accelerator Co-Design	HPCA	2023	https://doi.org/10.1109/HPCA56546.2023.10071027
AutoBridge (FPGA 2021)	AutoBridge: Coupling Coarse-Grained Floorplanning and Pipelining for High-Frequency HLS Design on Multi-Die	FPGA	2021	https://doi.org/10.1145/3431920.3439289
ARIES (FPGA 2025)	ARIES: An Agile MLIR-Based Compilation Flow for Reconfigurable Devices with AI Engines	FPGA	2025	https://doi.org/10.1145/3706628.3708870
Callipepla (FPGA 2023)	Callipepla: Stream Centric Instruction Set and Mixed Precision for Accelerating Conjugate Gradient Solver	FPGA	2023	https://doi.org/10.1145/3543622.3573182
Du et al (FPGA 2022)	High-Performance Sparse Linear Algebra on HBM-Equipped FPGAs Using HLS: A Case Study on SpMV	FPGA	2022	https://doi.org/10.1145/3490422.3502368

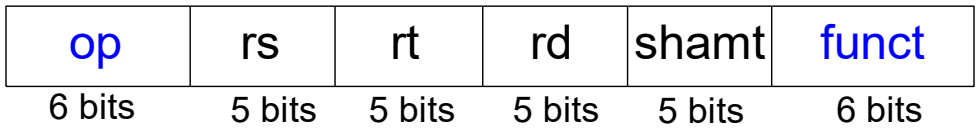
Single-Cycle Processor



R-Type

- *Register-type*
- 3 register operands:
  - `rs`, `rt`: source registers
  - `rd`: destination register
- Other fields:
  - `op`: the *operation code* or *opcode* (0 for R-type instructions)
  - `funct`: the *function* with opcode, tells computer what operation to perform
  - `shamt`: the *shift amount* for shift instructions, otherwise it's 0

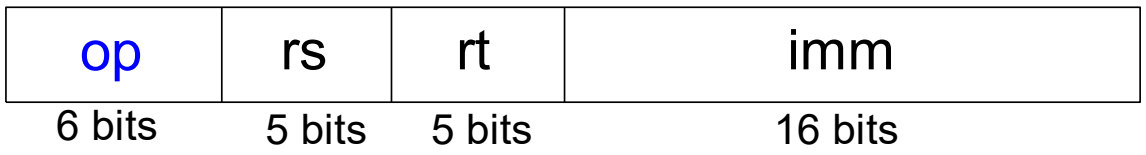
R-Type



I-Type

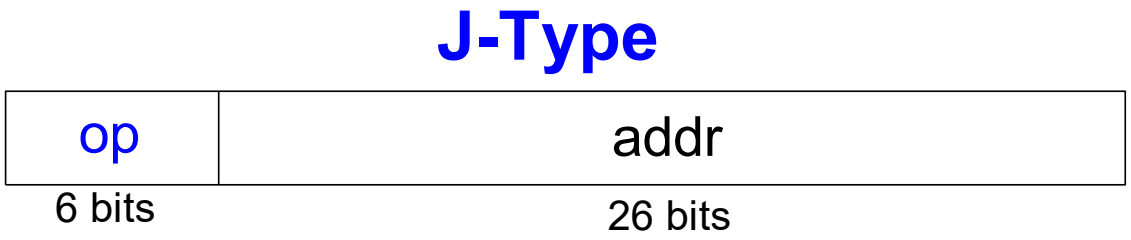
- *Immediate-type*
- 3 operands:
  - `rs`, `rt`: register operands
  - `imm`: 16-bit two's complement immediate
- Other fields:
  - `op`: the opcode
  - Simplicity favors regularity: all instructions have opcode
  - Operation is completely determined by opcode

I-Type



Machine Language: J-Type

- *Jump-type*
- 26-bit address operand (`addr`)
- Used for jump instructions (`j`)

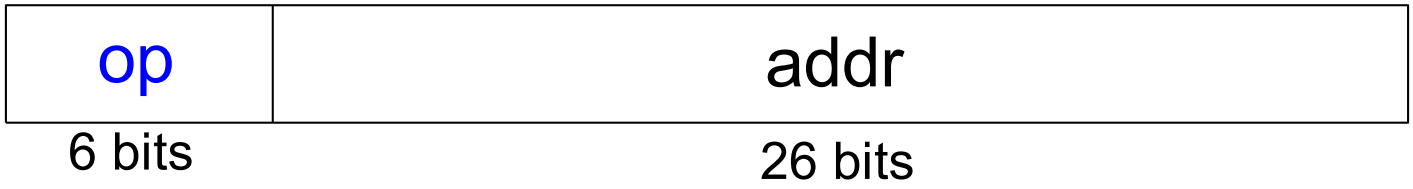


Review: Instruction Formats

R-Type

I-Type

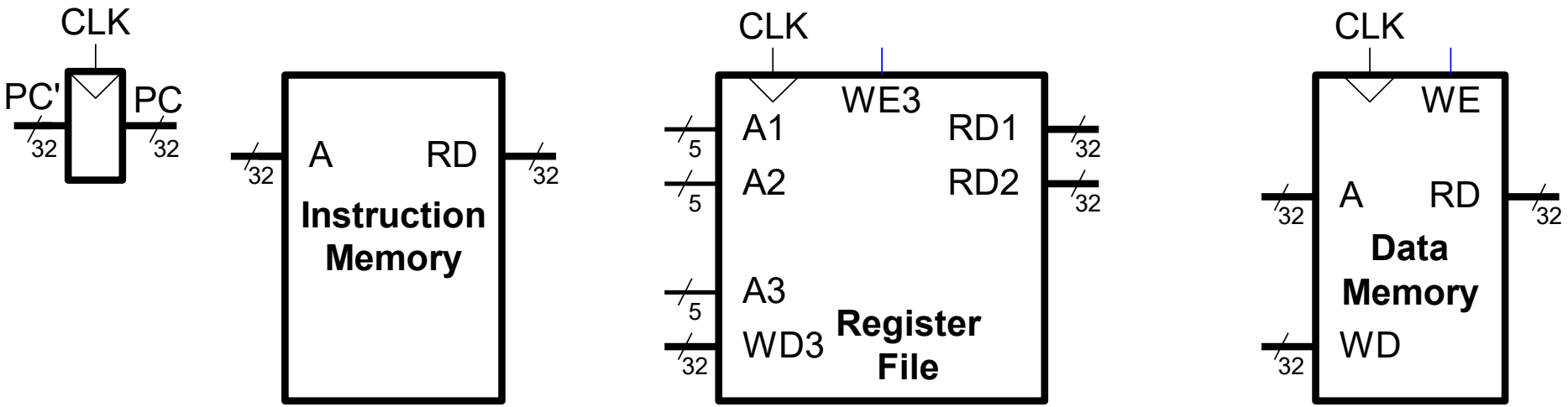
J-Type



Architectural State

- Determines everything about a processor:
  - PC
  - 32 registers
  - Memory

MIPS State Elements

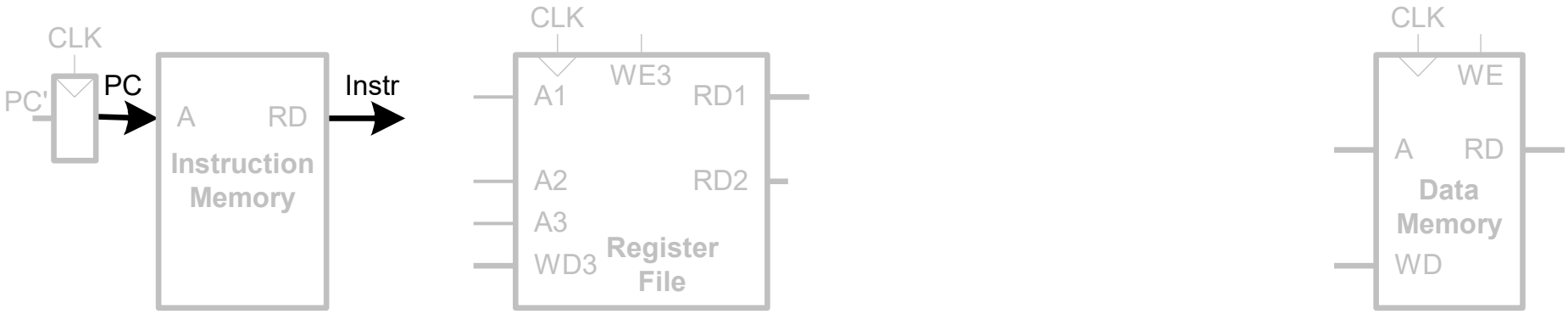


Single-Cycle MIPS Processor

- Datapath
- Control

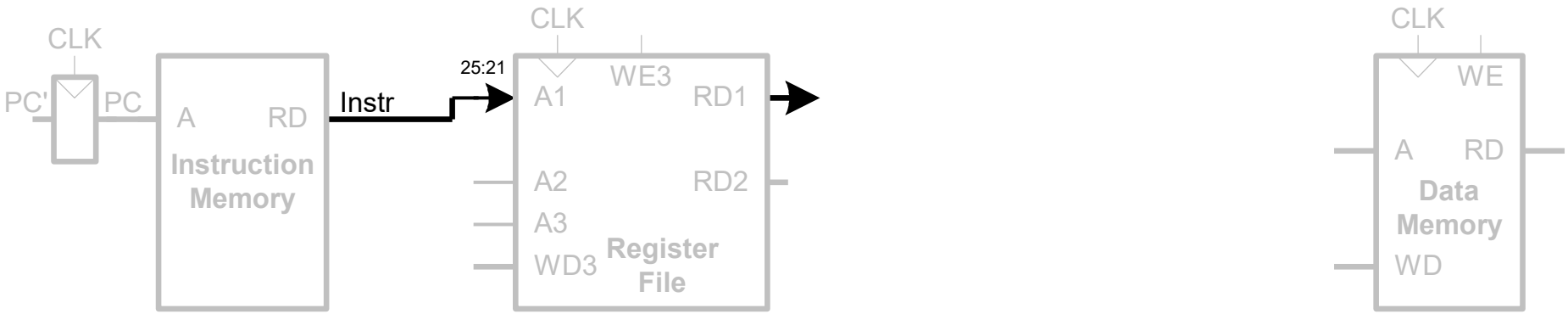
Single-Cycle Datapath: `lw` fetch

STEP 1: Fetch instruction



Single-Cycle Datapath: `lw` Register Read

STEP 2: Read source operands from RF



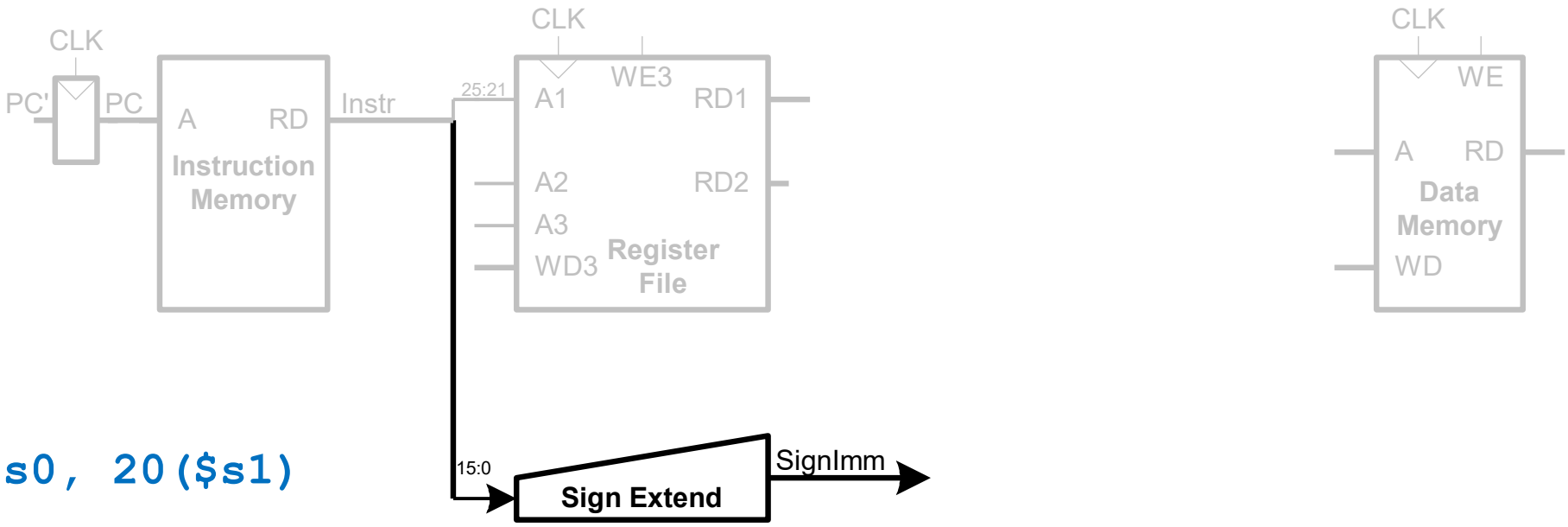

```
lw $s0, 20($s1)
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: `lw` Immediate

STEP 3: Sign-extend the immediate



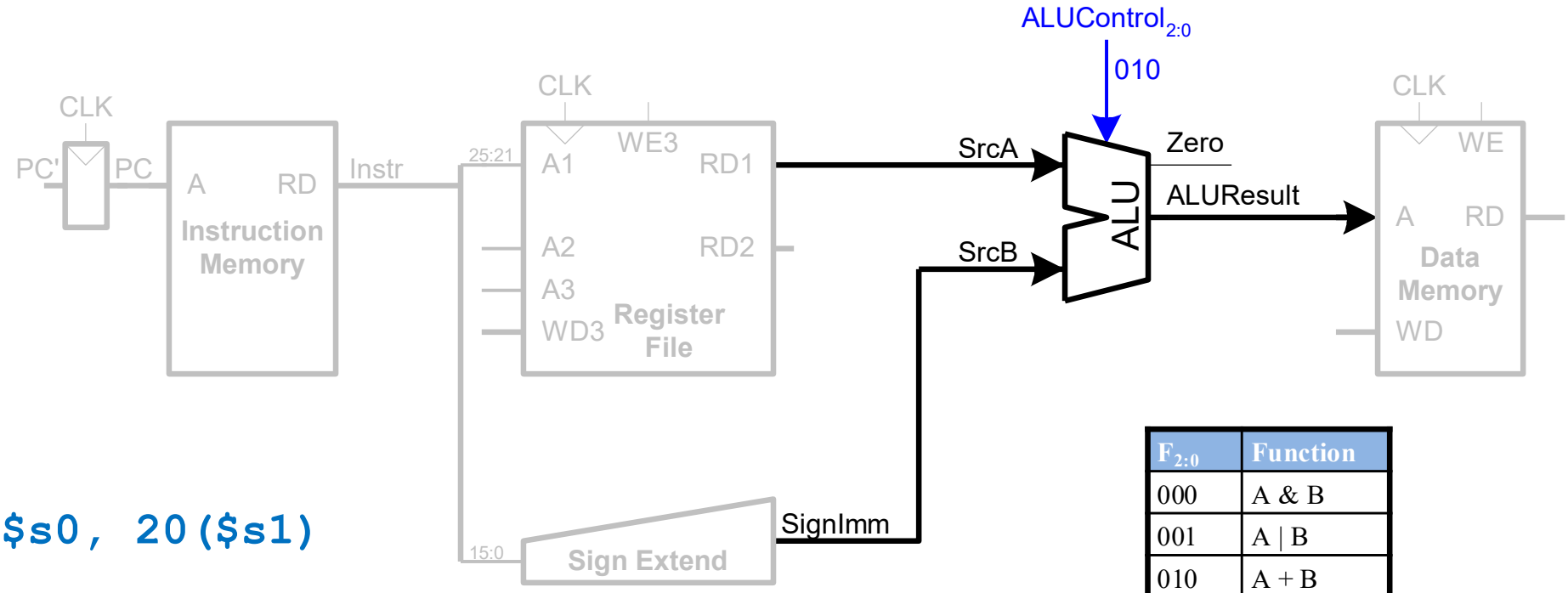

```
lw $s0, 20($s1)
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

Single-Cycle Datapath: `lw` address

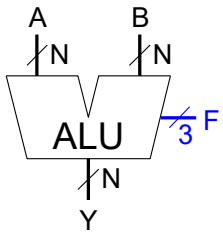
STEP 4: Compute the memory address




```
lw $s0, 20($s1)
```

I-Type

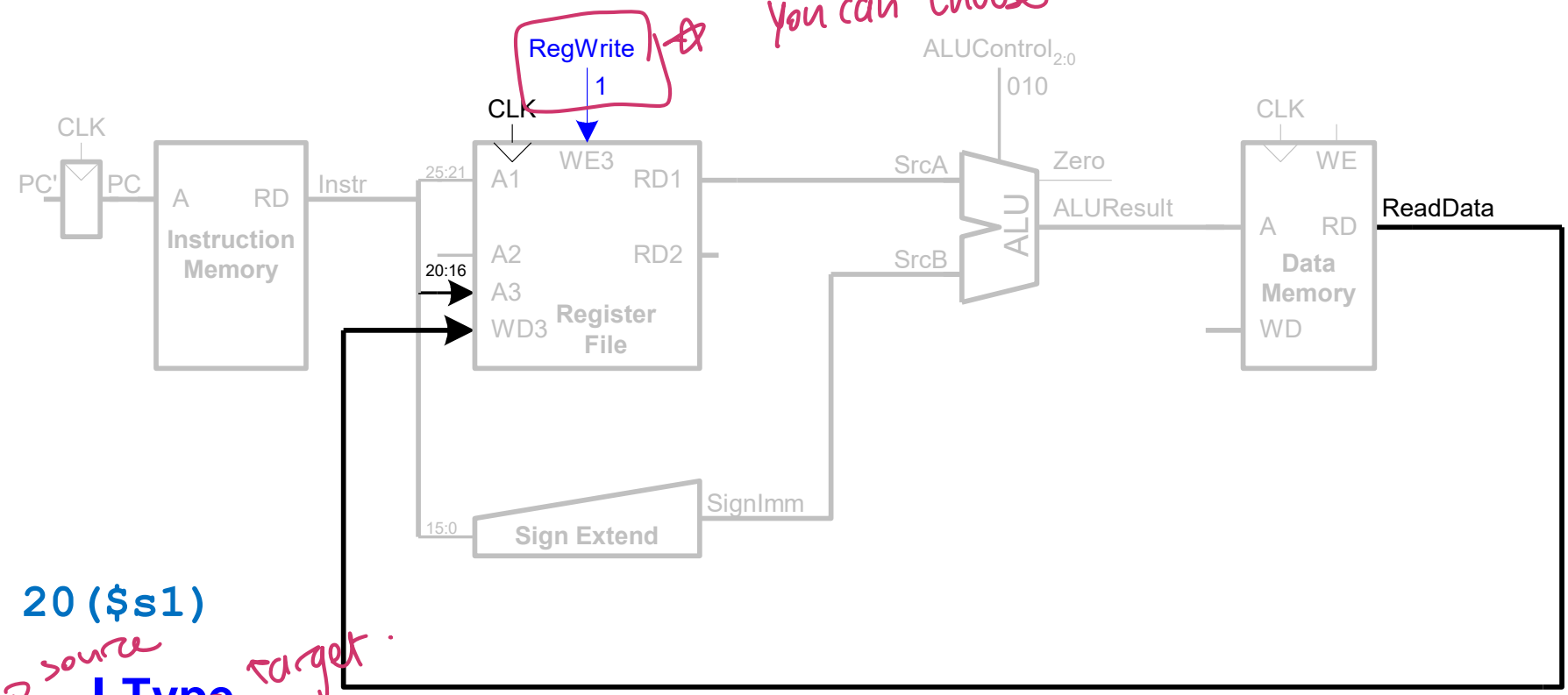
op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

Single-Cycle Datapath: `lw` Memory Read

- **STEP 5:** Read data from memory and write it back to register file




```
lw $s0, 20($s1)
```

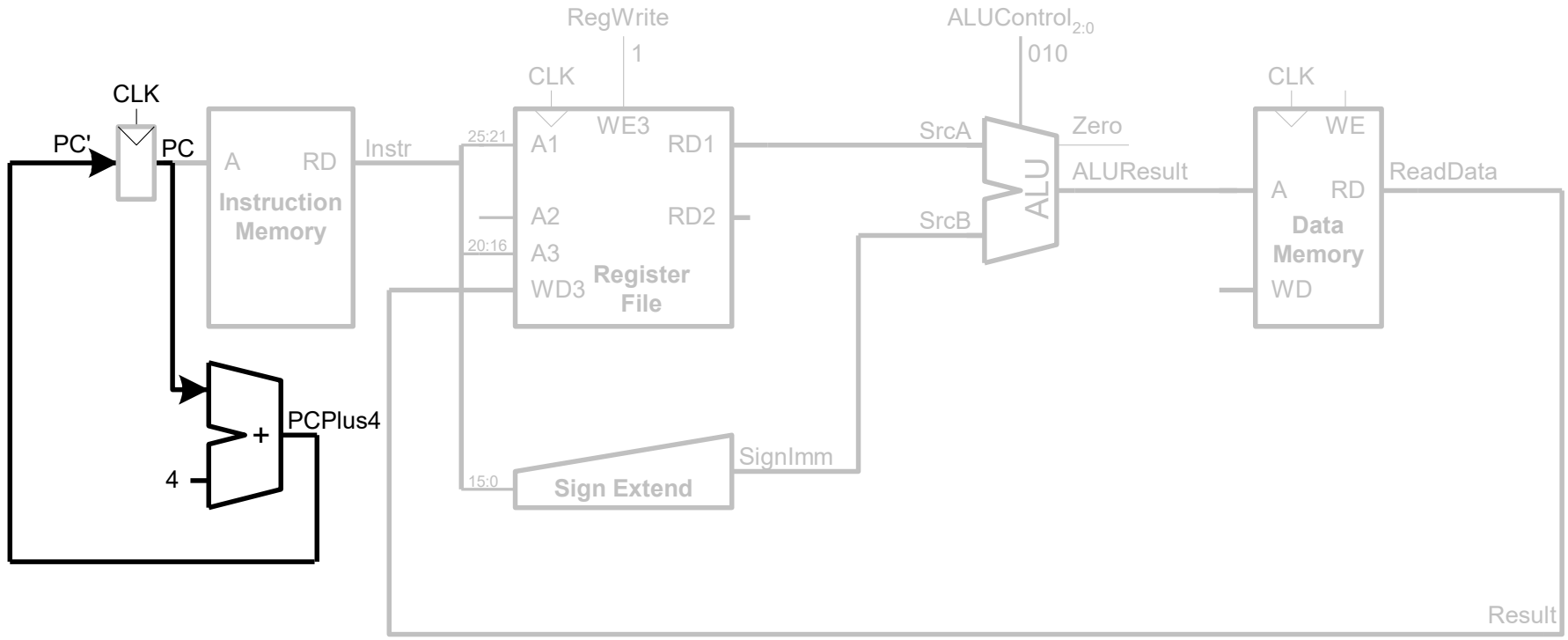
source target

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

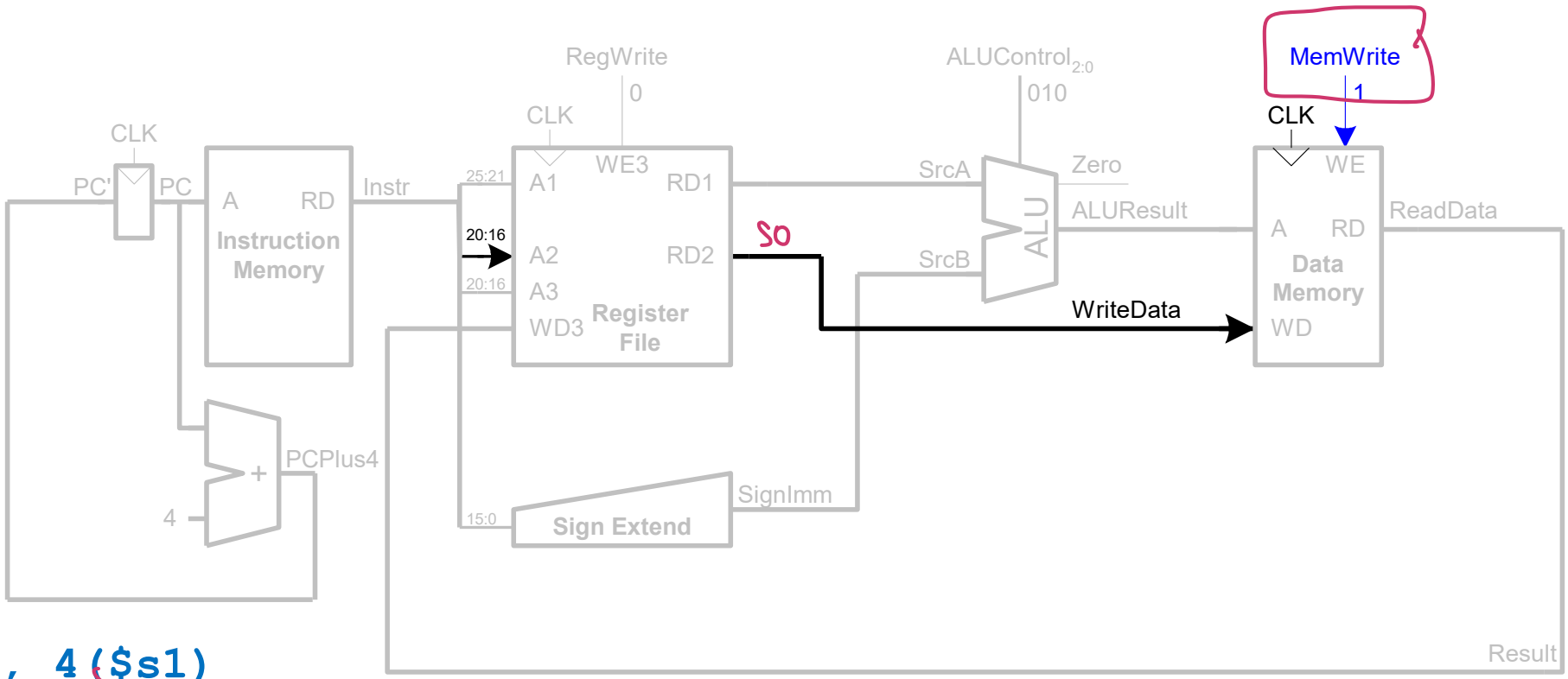
Single-Cycle Datapath: `lw` PC Increment

STEP 6: Determine address of next instruction



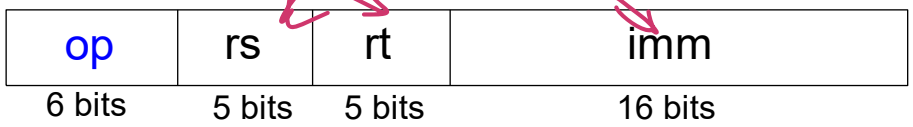
Single-Cycle Datapath: sw

Write data in rt to memory



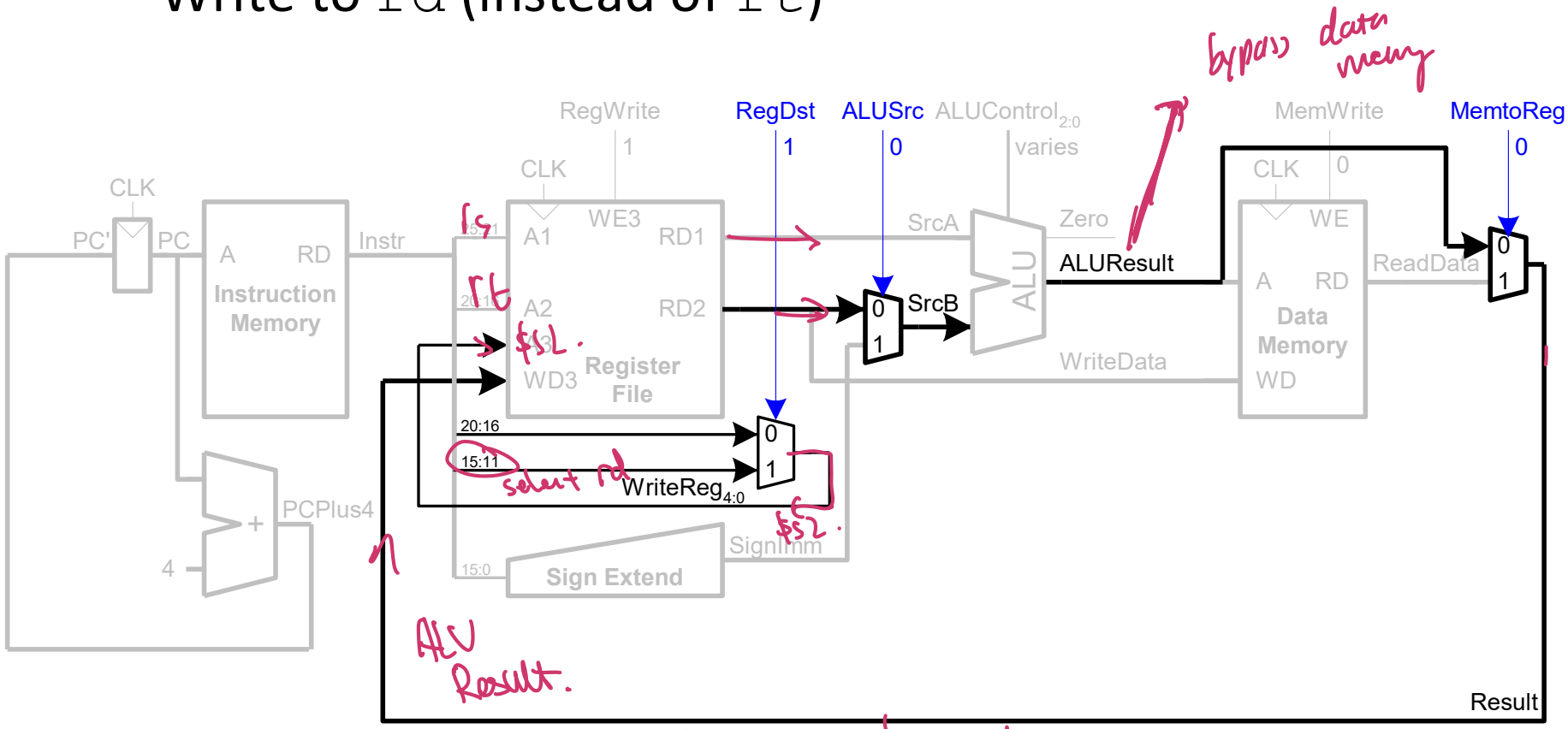

```
sw $s0, 4($s1)
```

I-Type



Single-Cycle Datapath: R-Type

- Read from `rs` and `rt`
- Write *ALUResult* to register file
- Write to `rd` (instead of `rt`)




```
add  $s2, $s1, $s0
```

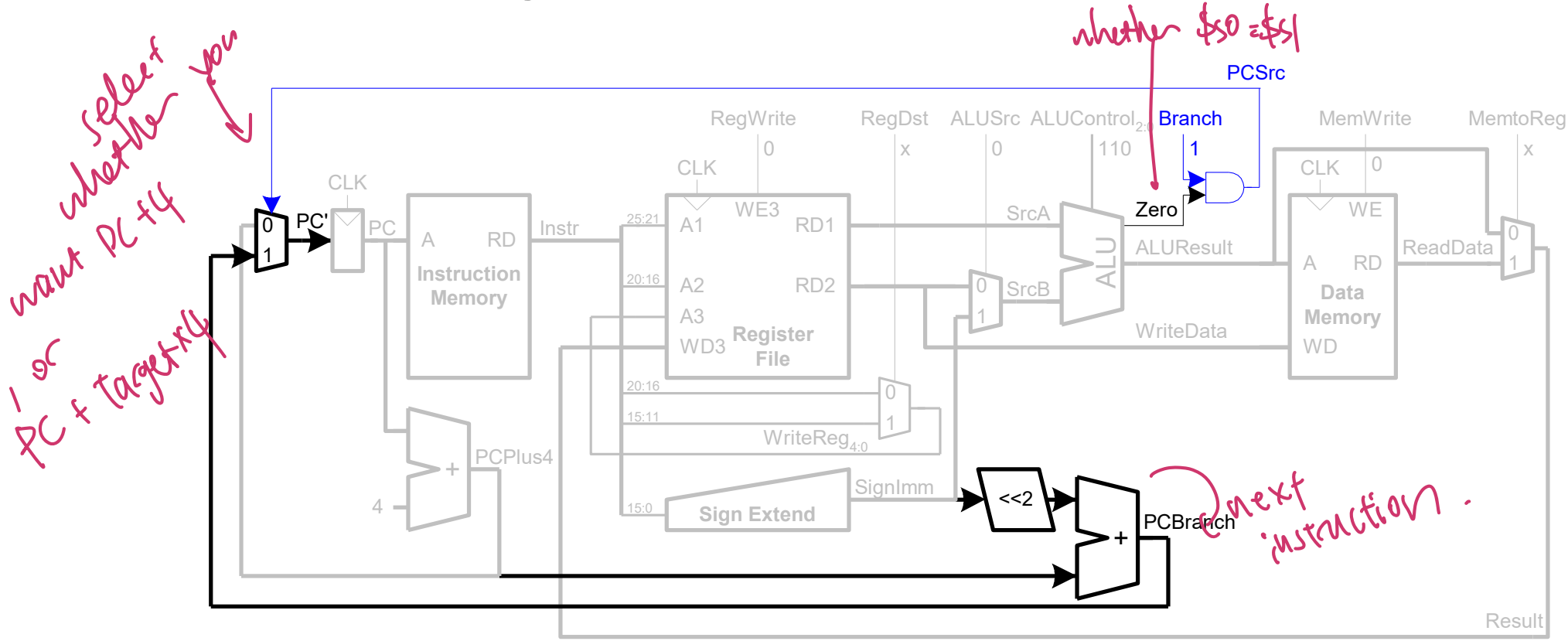
R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Single-Cycle Datapath: beq

- Determine whether values in rs and rt are equal
- Calculate branch target address:

  BTA = (sign-extended immediate << 2) + (PC+4)

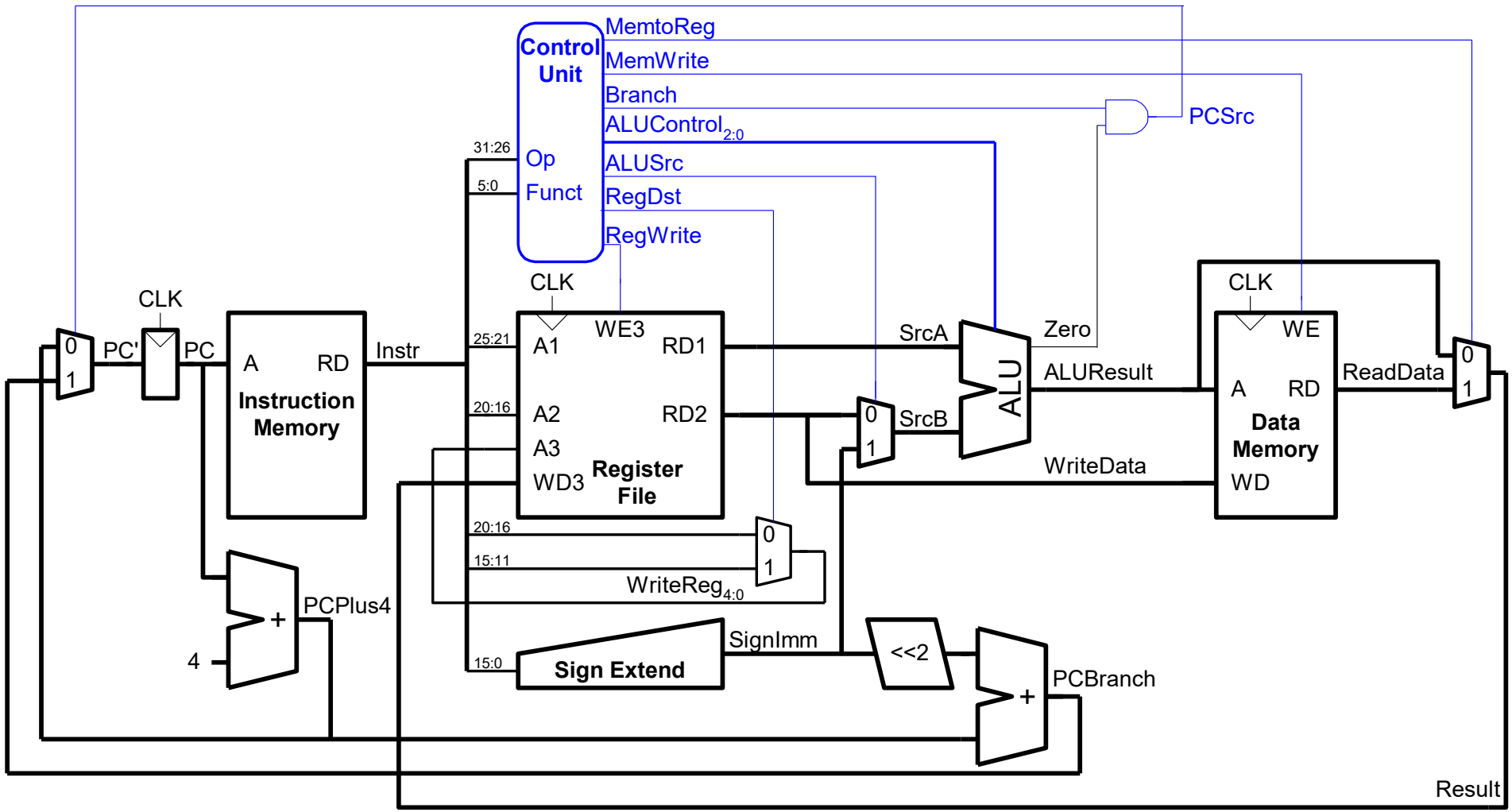



```
beq  $s0, $s1, target
```

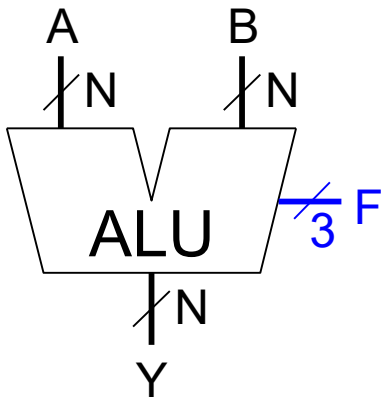
I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

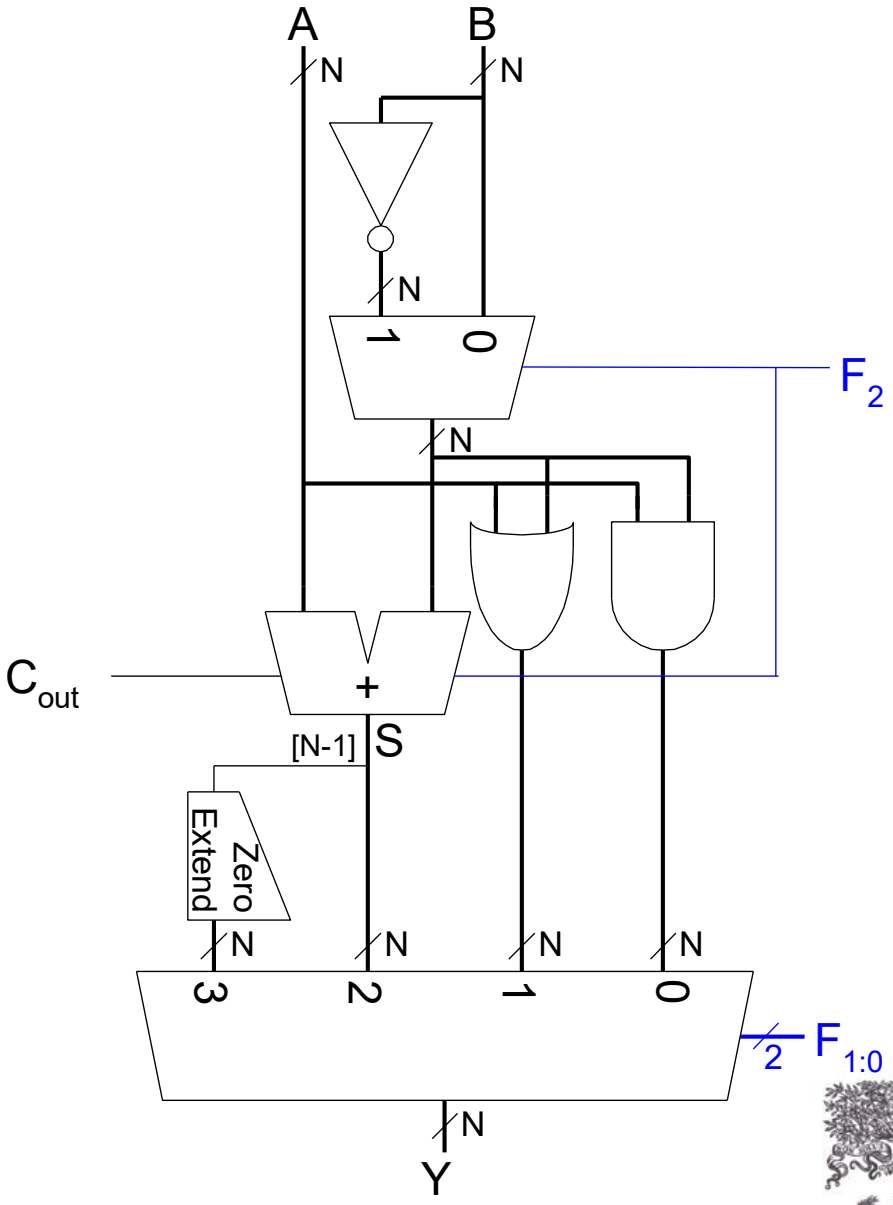
Single-Cycle Processor



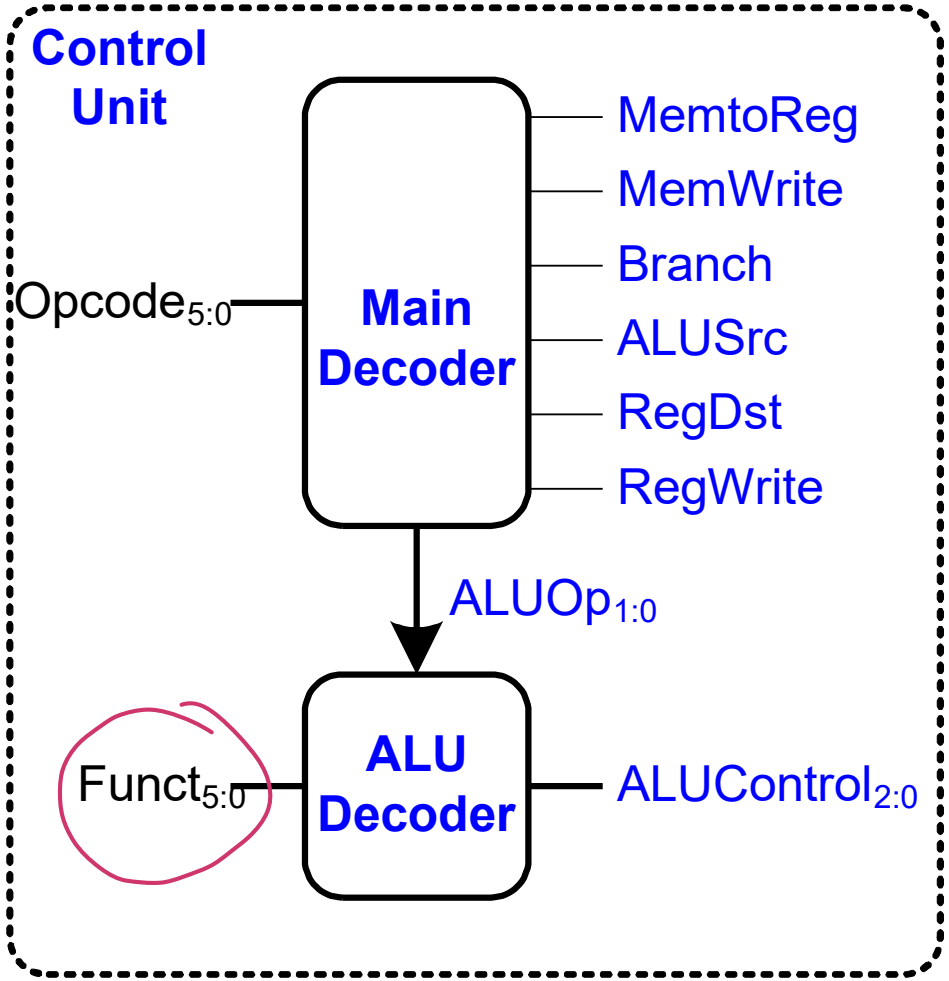
Review: ALU



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT



Control Unit & ALU Decoder



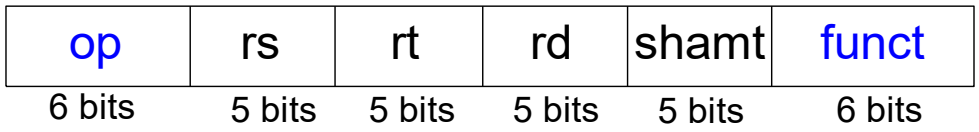
$ALUOp_{1:0}$	Meaning
00	Add
01	Subtract
10	Look at Funct
11	Not Used

$ALUOp_{1:0}$	Funct	$ALUControl_{2:0}$
00	X	010 (Add)
X1	X	110 (Subtract)
1X	100000 (add)	010 (Add)
1X	100010 (sub)	110 (Subtract)
1X	100100 (and)	000 (And)
1X	100101 (or)	001 (Or)
1X	101010 (slt)	111 (SLT)

R-Type

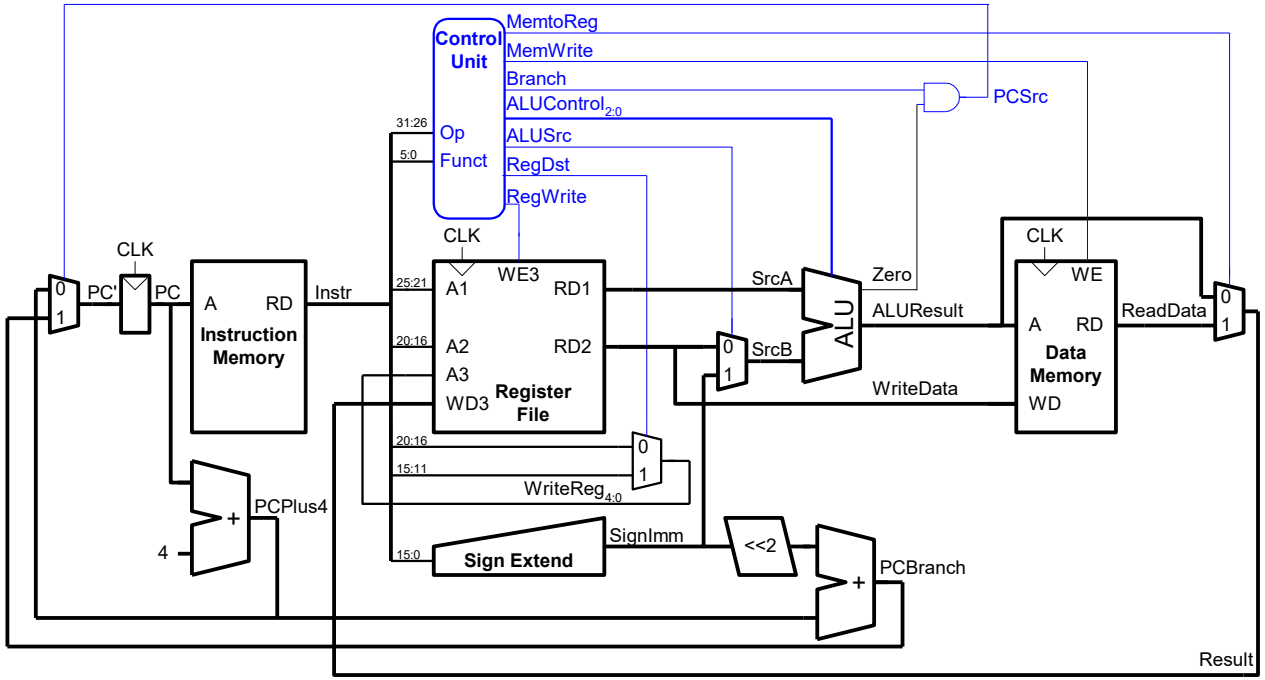
- *Register-type*
- 3 register operands:
  - `rs`, `rt`: source registers
  - `rd`: destination register
- Other fields:
  - `op`: the *operation code* or *opcode* (0 for R-type instructions)
  - `funct`: the *function* with opcode, tells computer what operation to perform
  - `shamt`: the *shift amount* for shift instructions, otherwise it's 0

R-Type



Control Unit Main Decoder

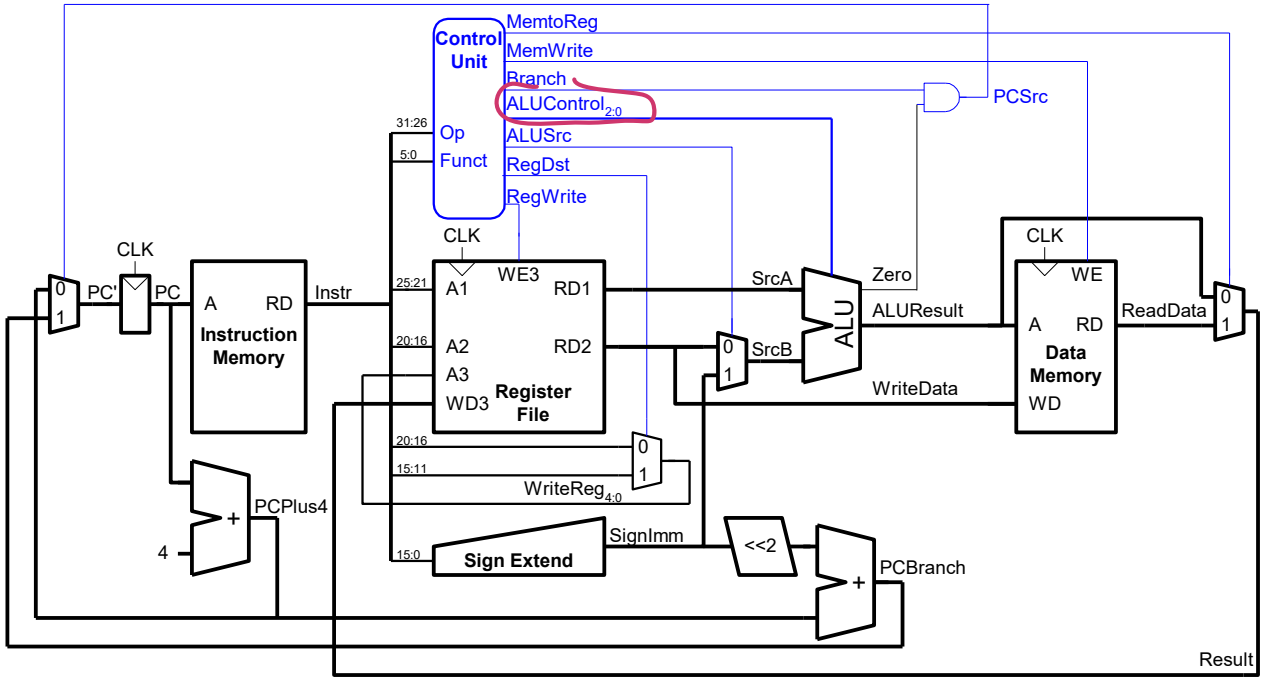
Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000							
`lw`	100011							
`sw`	101011							
`beq`	000100							



Control Unit Main Decoder

look at function.

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011							
`sw`	101011							
`beq`	000100							



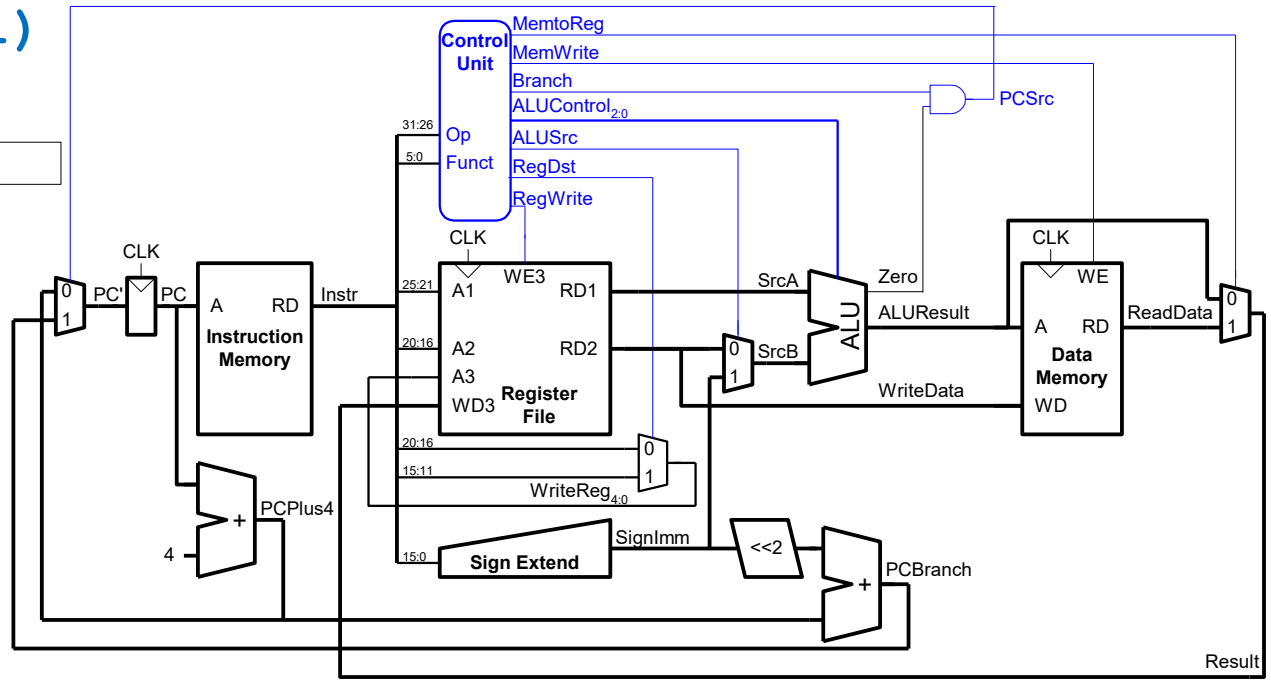
Control Unit Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011	1	0	1	0	0	1	00 ← add base & $s1
`sw`	101011							
`beq`	000100							

```
lw $s0, 20($s1)
```

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits



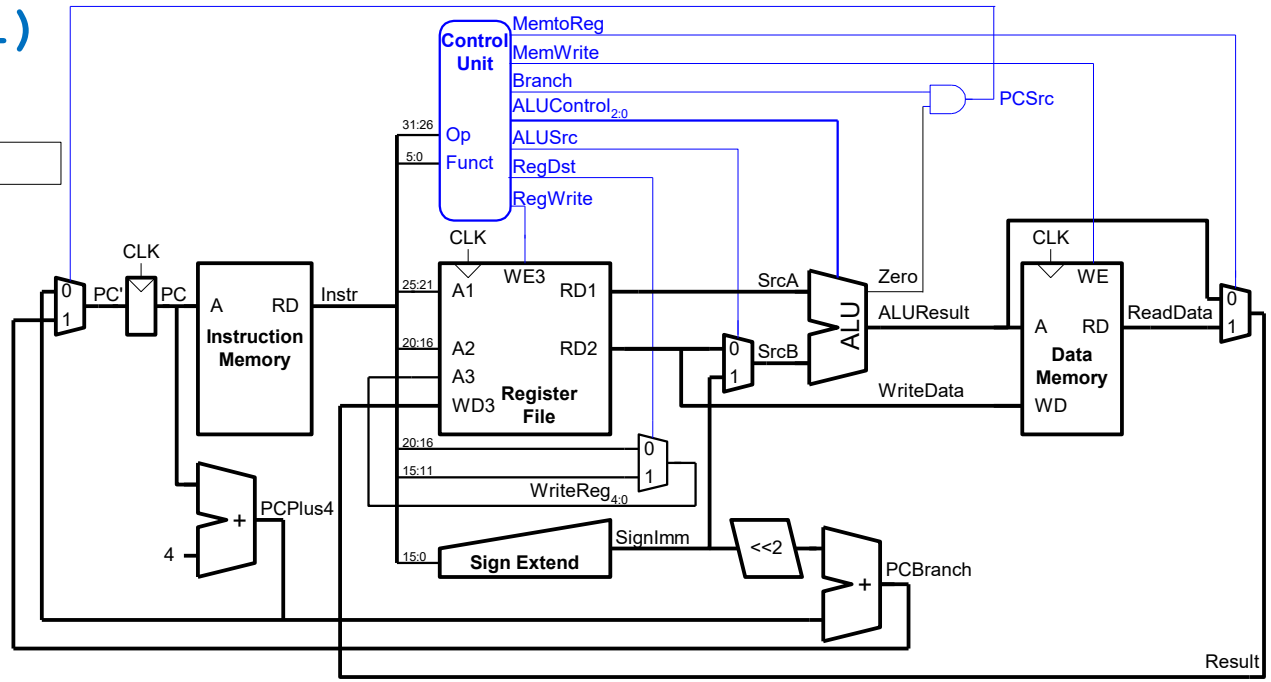
Control Unit Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011	1	0	1	0	0	0	00
`sw`	101011							
`beq`	000100							

`lw $s0, 20($s1)`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits



Control Unit Main Decoder

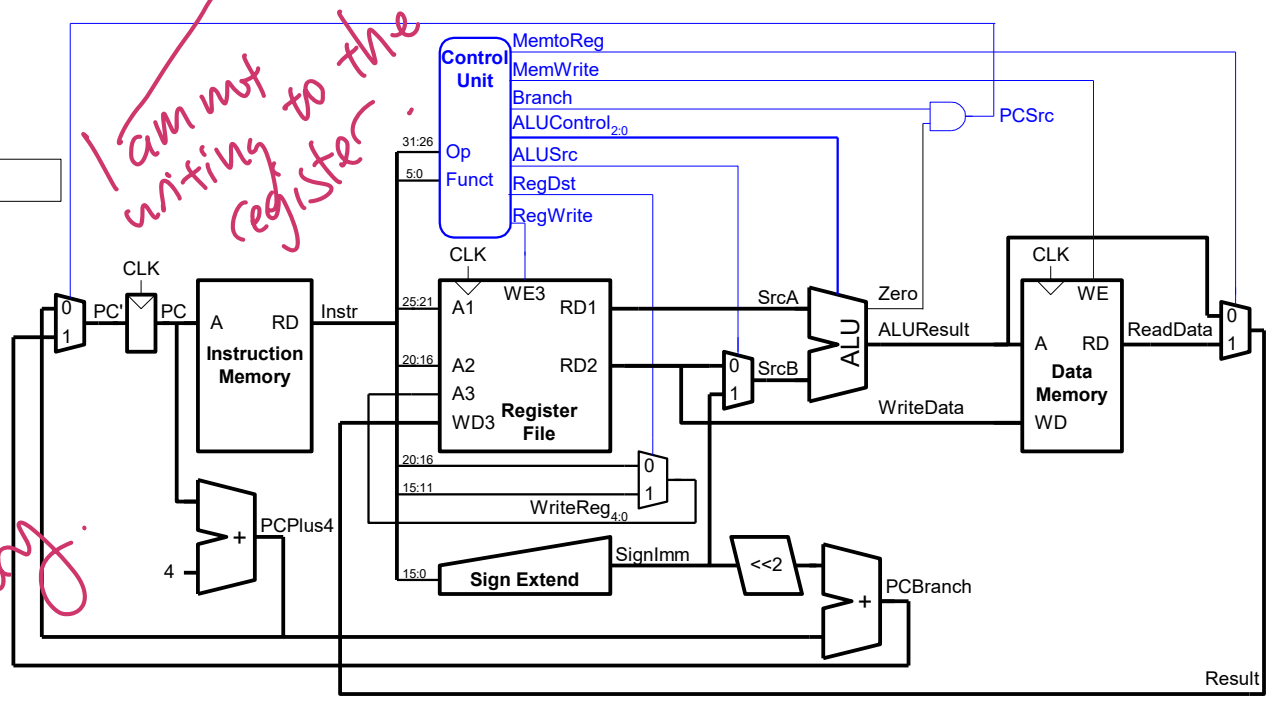
Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011	1	0	1	0	0	~~0~~ 1	00
`sw`	101011	0	X	1	0	1	X	00
`beq`	000100							

`sw $s0, 4($s1)`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

I am not writing to the register.



Store val in $s0 to 4($s1) in data memory.

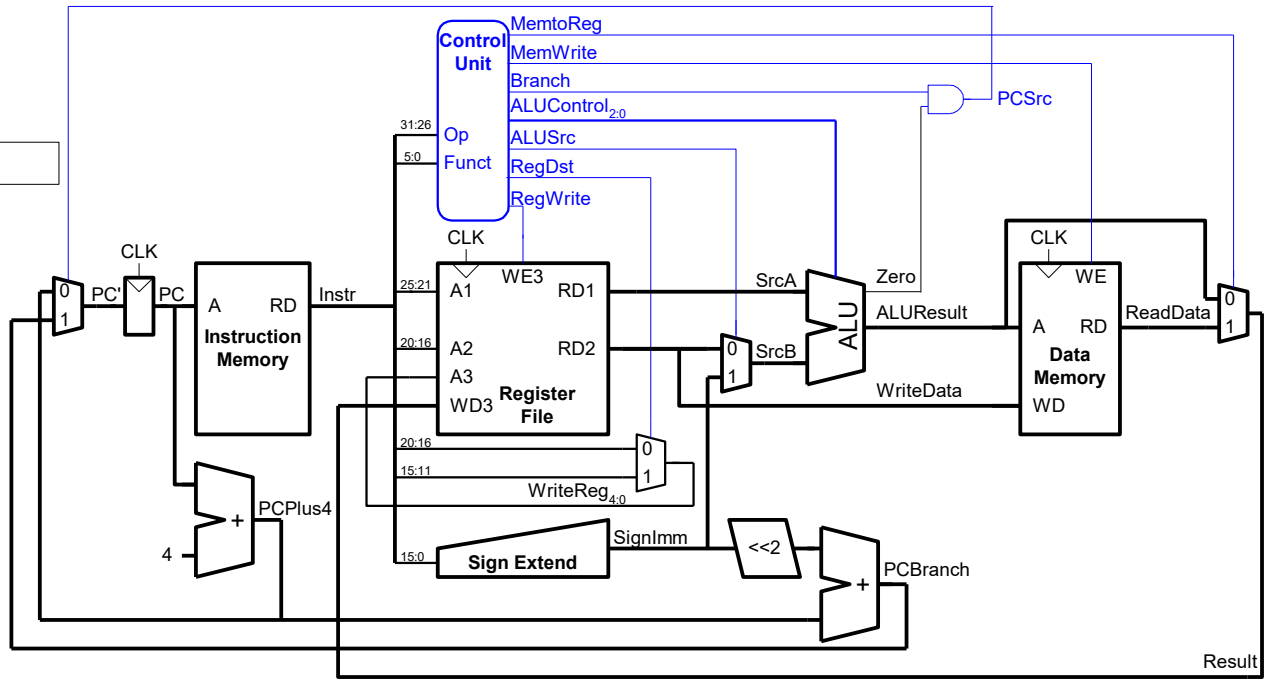
Control Unit Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011	1	0	1	0	0	0	00
`sw`	101011	0	X	1	0	1	X	00
`beq`	000100							

`sw $s0, 4($s1)`

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits



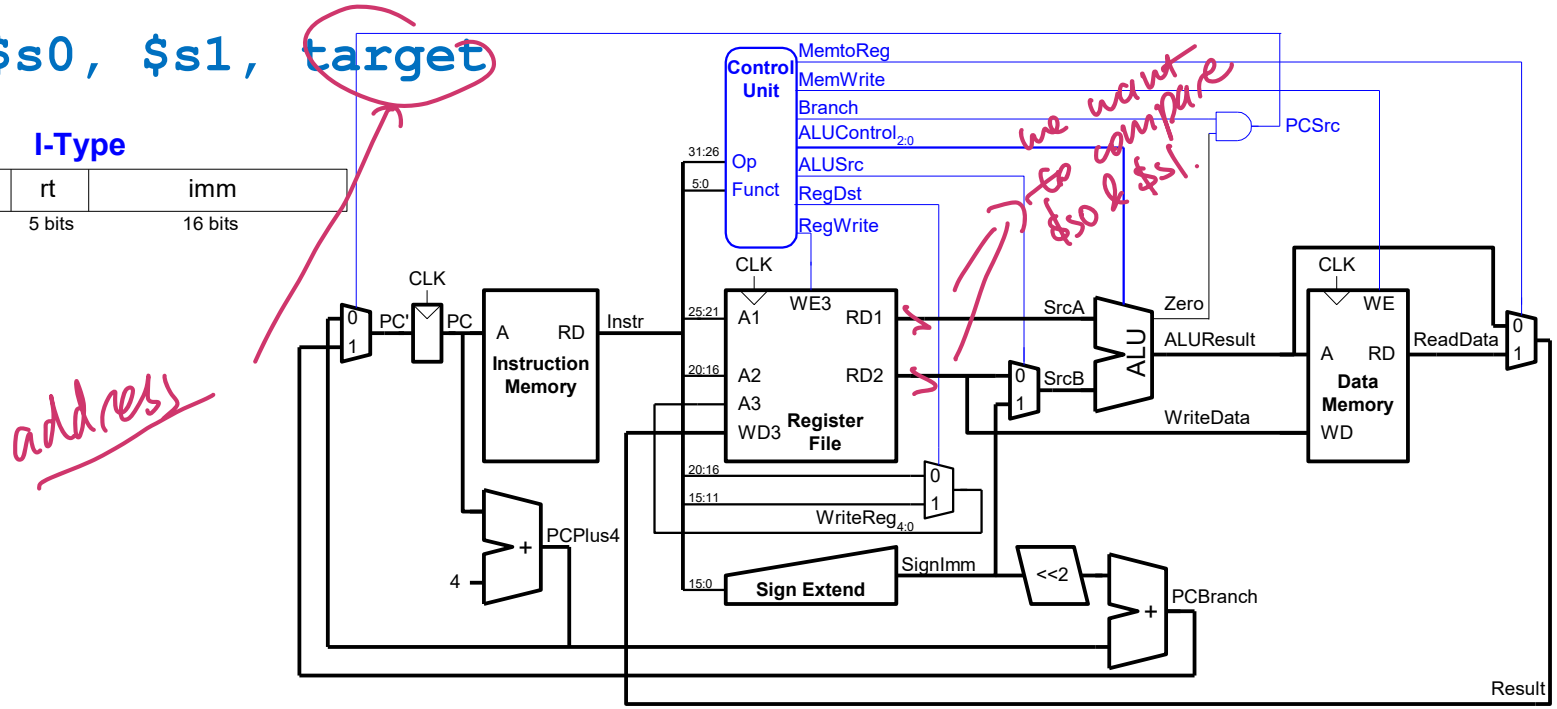
Control Unit Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011	1	0	1	0	0	0	00
`sw`	101011	0	X	1	0	1	X	00
`beq`	000100	0	X	0	1	X	X	01

```
beq $s0, $s1, target
```

I-Type

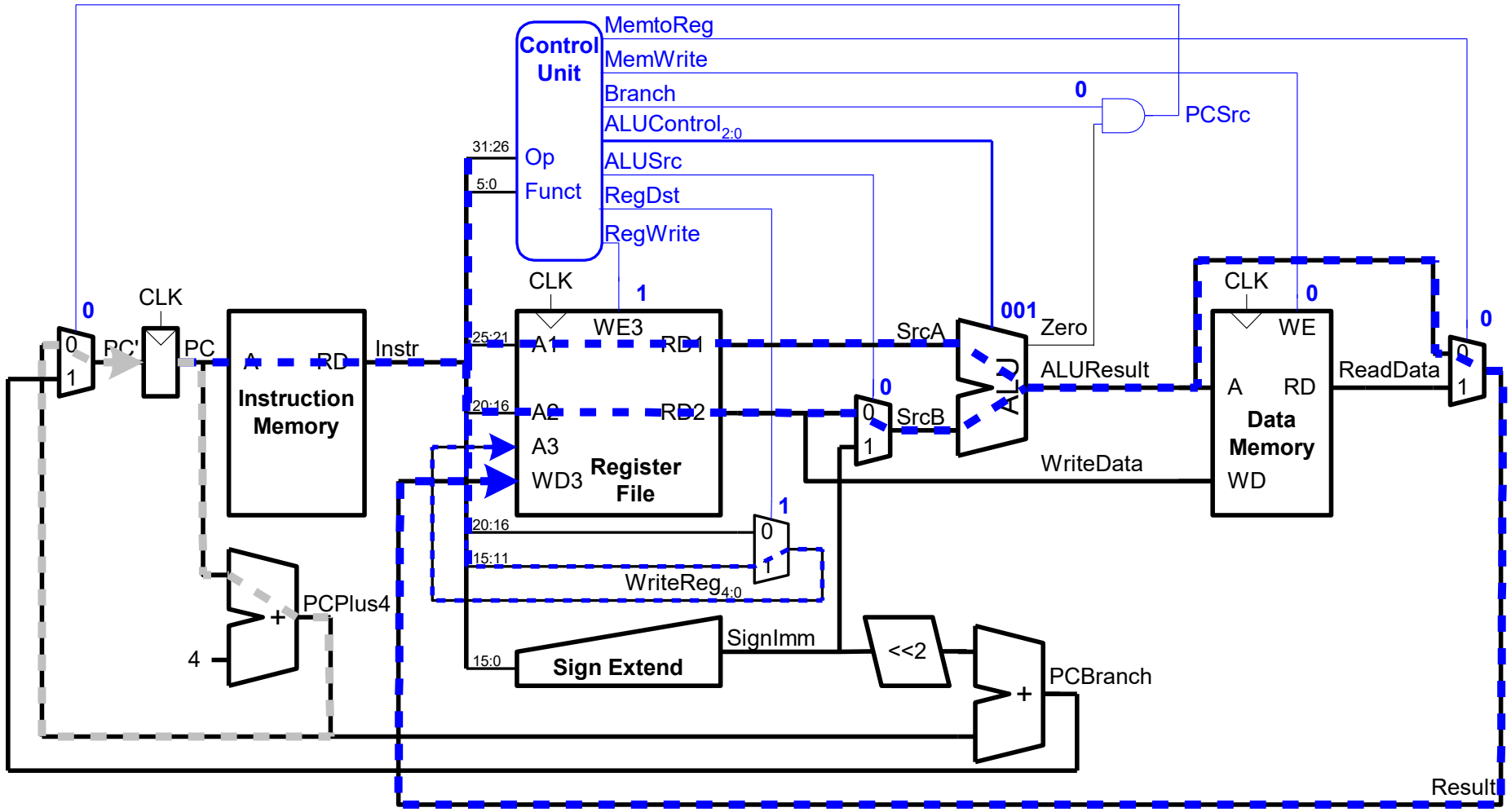
op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits



Control Unit: Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
`lw`	100011	1	0	1	0	0	~~0~~ 1	00
`sw`	101011	0	X	1	0	1	X	00
`beq`	000100	0	X	0	1	0	X	01

Single-Cycle Datapath: or

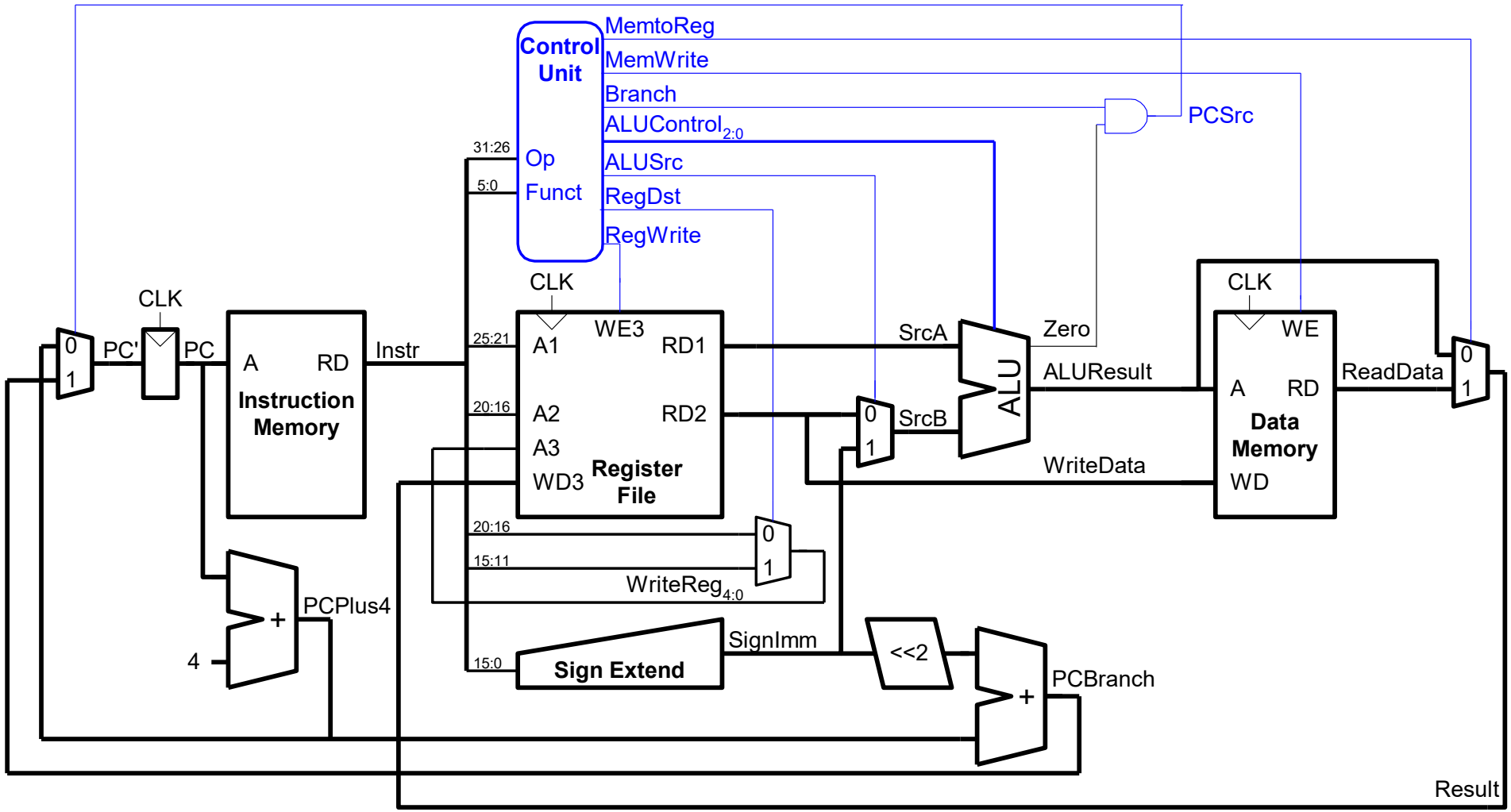



```
or $t0, $t1, $t2
```

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Extended Functionality: addi



No change to datapath

Control Unit: addi

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	**001000**							

Control Unit: addi

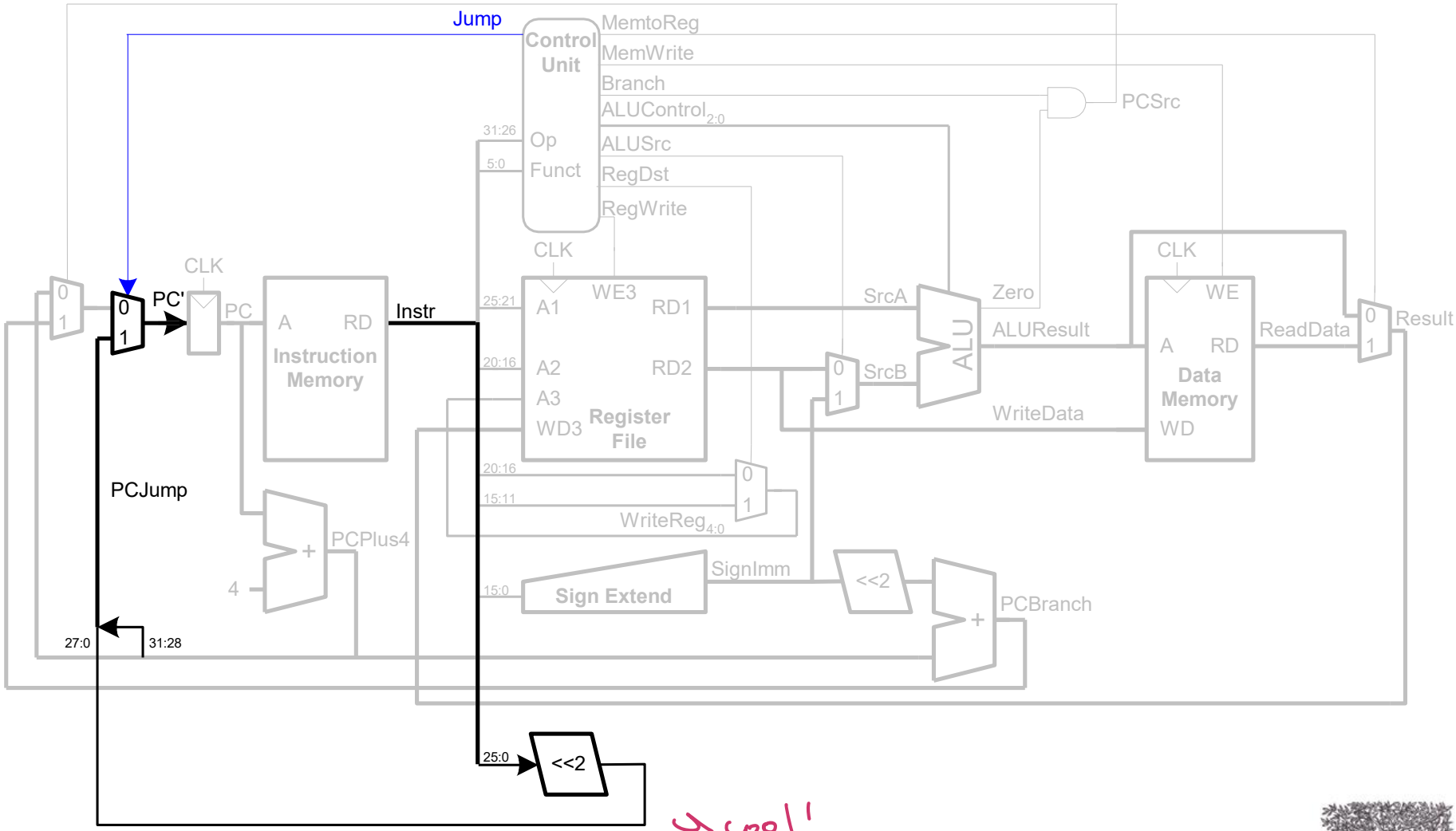
Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$
R-type	000000	1	1	0	0	0	0	10
lw	100011	1	0	1	0	0	1	00
sw	101011	0	X	1	0	1	X	00
beq	000100	0	X	0	1	0	X	01
addi	**001000**	**1**	**0**	**1**	**0**	**0**	**0**	**00**

addi, $rt, $rs, im

add val of $rs and im.

store value in $rt.

Extended Functionality: j



خوب!

Control Unit: Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$	Jump
R-type	000000	1	1	0	0	0	0	10	0
`lw`	100011	1	0	1	0	0	1	00	0
`sw`	101011	0	X	1	0	1	X	00	0
`beq`	000100	0	X	0	1	0	X	01	0
`j`	000010								

Control Unit: Main Decoder

Instruction	$Op_{5:0}$	RegWrite	RegDst	AluSrc	Branch	MemWrite	MemtoReg	$ALUOp_{1:0}$	Jump
R-type	000000	1	1	0	0	0	0	10	0
`lw`	100011	1	0	1	0	0	1	00	0
`sw`	101011	0	X	1	0	1	X	00	0
`beq`	000100	0	X	0	1	0	X	01	0
`j`	000010	0	X	X	X	0	X	XX	1

data safety